

<https://www.vbtutor.net/vb2008/vb2008tutor.html>

### **Color.FromArgb( RGB codes).**

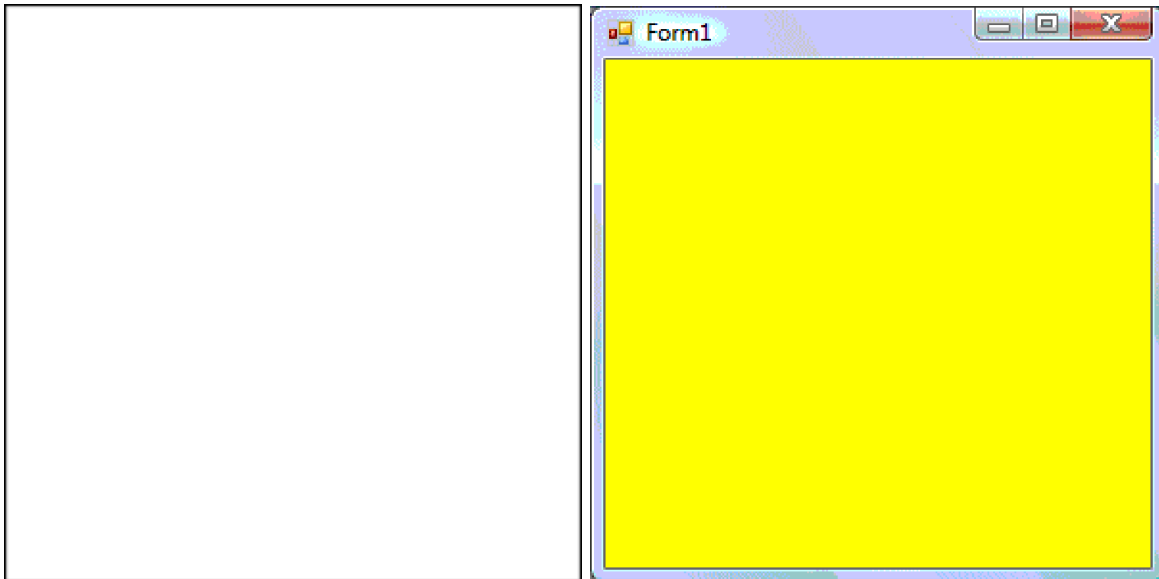
```
Public Class Form1
    Private Sub Form1_Load(ByVal sender As System.Object, ByVal e As
System.EventArgs) Handles MyBase.Load
        Me.BackColor = Color.FromArgb(255, 255, 0)
    End Sub
End Class
```

End Sub  
End Class

You may also use the follow procedure to assign the color at run time.

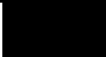
```
Private Sub Form1_Load(ByVal sender As System.Object, ByVal e As
System.EventArgs) Handles MyBase.Load
    Me.BackColor = Color.Yellow
End Sub
```

Both procedures above will load the form with a yellow background as shown in Figure 3.3.

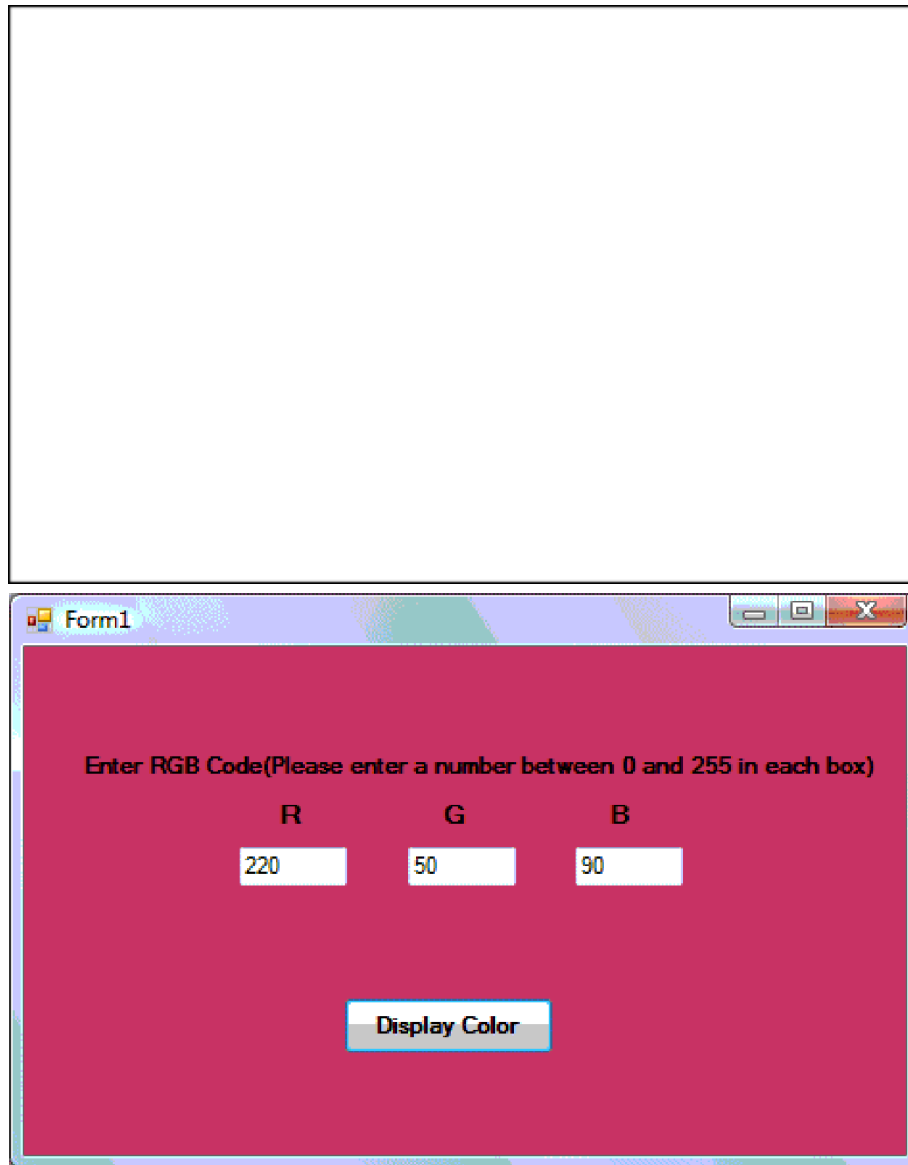


The following table shows some of the common colors and the corresponding RGB codes. You can always experiment with other combinations, but remember the maximum number for each color is 255 and the minimum number is 0.

| Color | RGB code | Color  | RGB code    | Color  | RGB Code    |
|-------|----------|--------|-------------|--------|-------------|
| Red   | 255,0,0  | Yellow | 255, 255, 0 | Orange | 255, 165, 0 |

|                                                                                   |           |                                                                                   |                |                                                                                   |               |
|-----------------------------------------------------------------------------------|-----------|-----------------------------------------------------------------------------------|----------------|-----------------------------------------------------------------------------------|---------------|
|  | 0,255,0   |  | 0, 255,<br>255 |  | 0, 0, 0       |
|  | 0, 0, 255 |  | 255, 0,<br>255 |  | 255, 255, 255 |

The following is another program that allows the user to enter the RGB codes into three different textboxes and when he/she clicks the display color button, the background color of the form will change according to the RGB codes, as shown in Figure 3.4. Hence, this program allows users to change the color properties of the form at run time.



## The code

```
Private Sub Button1_Click(ByVal sender As System.Object, ByVal e As
System.EventArgs) Handles Button1.Click
    Dim rgb1, rgb2, rgb3 As Integer
```

```

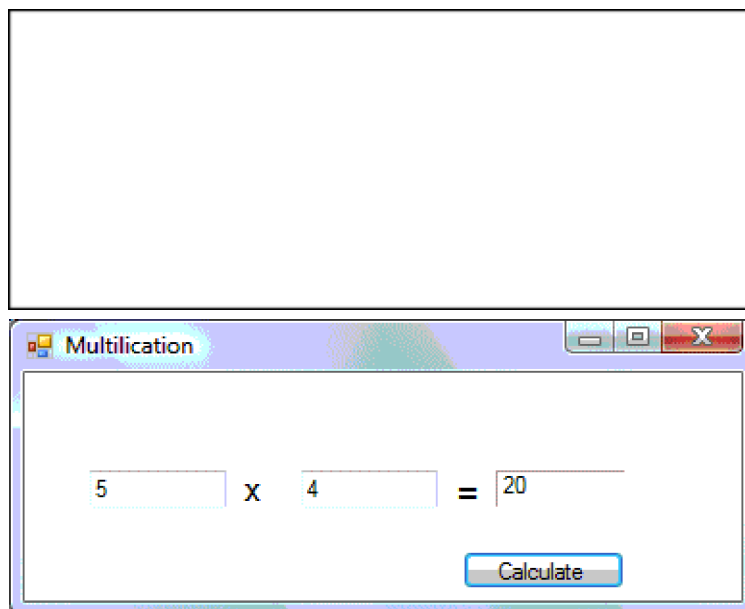
    rgb1 = TextBox1.Text
    rgb2 = TextBox2.Text
    rgb3 = TextBox3.Text
    Me.BackColor = Color.FromArgb(rgb1, rgb2, rgb3)
End Sub

```

End Sub

## **Using Text Box-A multiplication program**

In this program, you insert two textboxes , three labels and one button. The two textboxes are for the users to enter two numbers, one label is to display the multiplication operator and the other label is to display the equal sign. The last label is to display the answer.



### **The Code**

```

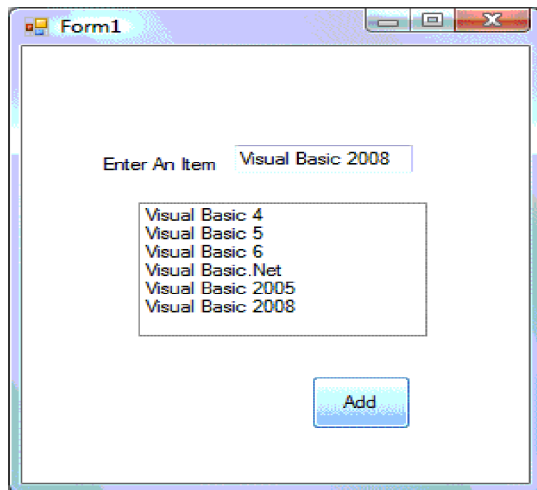
Private Sub Button1_Click(ByVal sender As System.Object, ByVal e As
System.EventArgs) Handles Button1.Click
    Dim num1, num2, product As Single
    num1 = TextBox1.Text
    num2 = TextBox2.Text
    product = num1 * num2
    Label3.Text = product
End Sub

```

## **2.2 Using the ListBox-A program to add items to a list box**

This program will add one item at a time as the user enter an item into the TextBox and

click the Add button.



## The code

```
Class Frm1
```

```
Private Sub Button1_Click(ByVal sender As System.Object, ByVal e As  
System.EventArgs) Handles Button1.Click
```

```
Dim item As String
```

```
item = TextBox1.Text
```

'To add items to a listbox

```
ListBox1.Items.Add(item)
```

```
End Sub
```

```
End Class
```

**Example1:** Write a program to print (hello) five times.

**Sol:**

```
Dim i as integer
```

```
Private Sub Command1_Click ()
```

```
For i = 1 To 5
```

```
Print "hello"
```

```
Next i
```

```
End Sub
```

Iterating Through an Array When you iterate through an array, you access each element in the array from the lowest index to the highest index. The following example iterates through a one-dimensional array by using the For...Next Statement (Visual Basic). The GetUpperBound method returns the highest value that the index can have. The lowest index value is always 0

```
Dim numbers = {10, 20, 30}
```

```
For index = 0 To numbers.GetUpperBound(0)
```

```
Debug.WriteLine(numbers(index))
```

```
Next
```

Output:

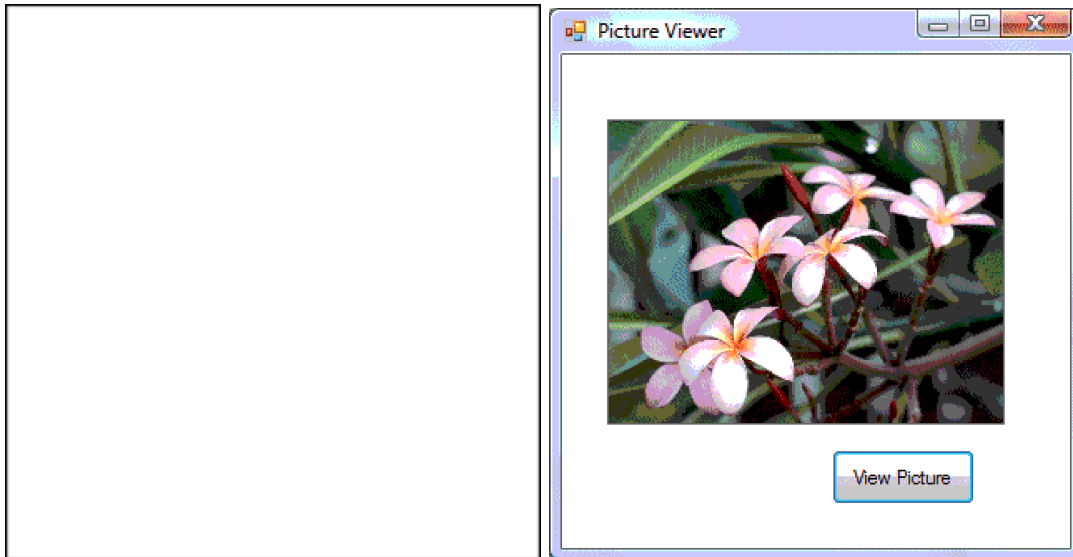
10

20

30

## 2.3 Using the PictureBox

In this program, we insert a PictureBox and a Button into the form. Make sure to set the SizeMode property of the PictureBox to StretchImage so that the whole picture can be viewed in the picture box. Key in the code as shown below and you can load an image from a certain image file into the PictureBox.



## The Code

```
Public Class Form1
```

```
Private Sub Button1_Click (ByVal sender As System.Object, ByVal e As  
System.EventArgs) Handles Button1.Click
```

```
'To load an image into the PictureBox from an image file
```

```
PictureBox1.Image = Image.FromFile("c:\Users\Public\Pictures\Sample  
Pictures\Frangipani Flowers.jpg")
```

```
End Sub
```

```
Private Sub Button1_Click_1(ByVal sender As System.Object, ByVal e As  
System.EventArgs) Handles Button1.Click
```

```
Dim name1, name2, name3 As String
```

```
name1 = "John"
```

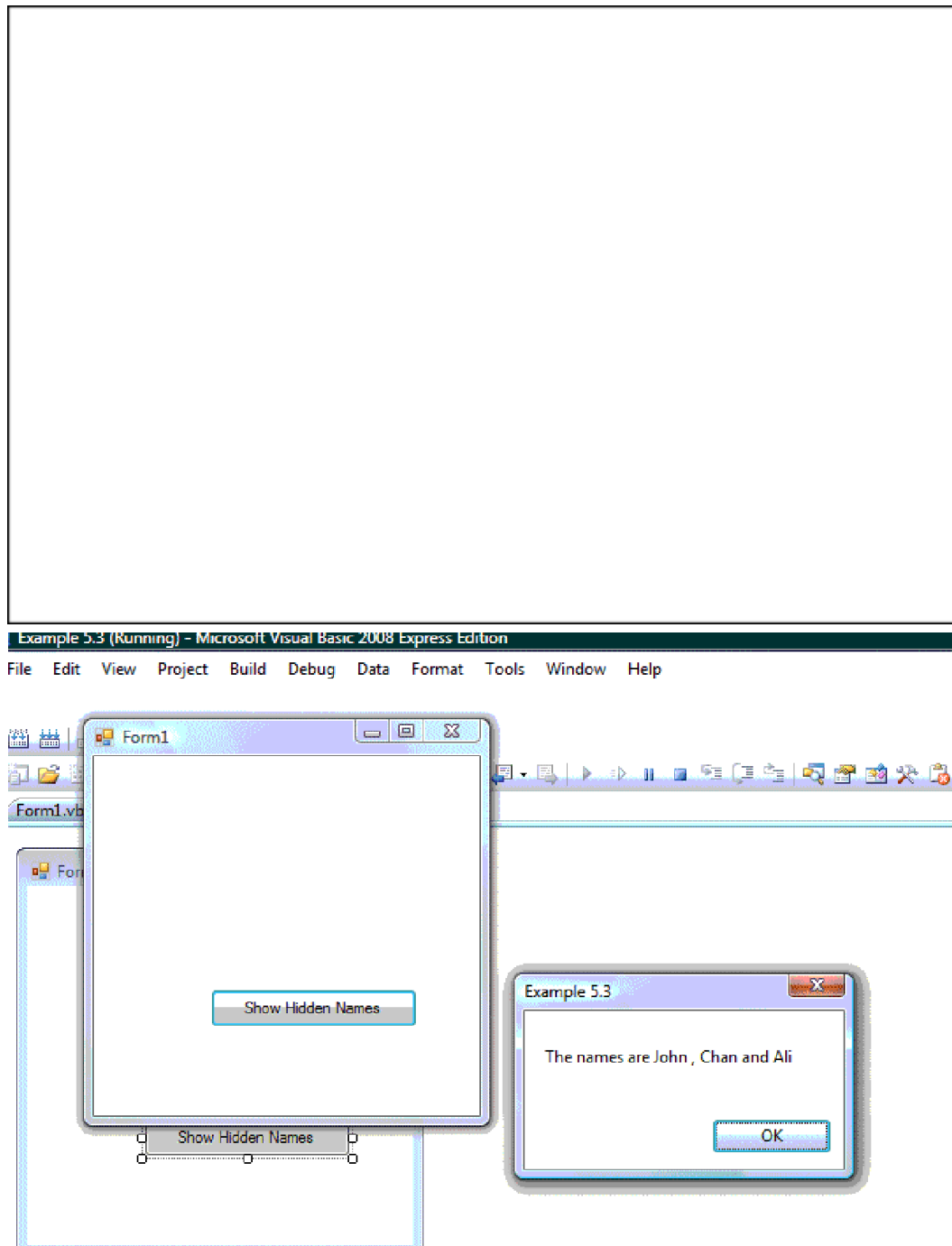
```
name2 = "Chan"
```

```
name3 = "Ali"
```

```
MsgBox(" The names are " & name1 & " , " & name2 & " and " & name3)
```

End Sub

In this example, you insert one command button into the form and rename its caption as Show Hidden Names. The keyword Dim is to declare variables name1, name2 and name3 as string, which means they can only handle text. The function MsgBox is to display the names in a message box that are joined together by the "&" signs. The output is shown in Figure 5.4.



**Private Sub** Form1\_Load(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles MyBase.Load



```
Me.Text="My First VB2008 Program"
```

```
Me.ForeColor = Color.Yellow
```

```
Me.BackColor = Color.Blue
```

```
End Sub
```



The output is shown in Figure 5.3.



## **Show hidden name**

```
Private Sub Button1_Click_1(ByVal sender As System.Object, ByVal e As  
System.EventArgs) Handles Button1.Click
```

```
Dim name1, name2, name3 As String
```

```
name1 = "John"
```

```
name2 = "Chan"
```

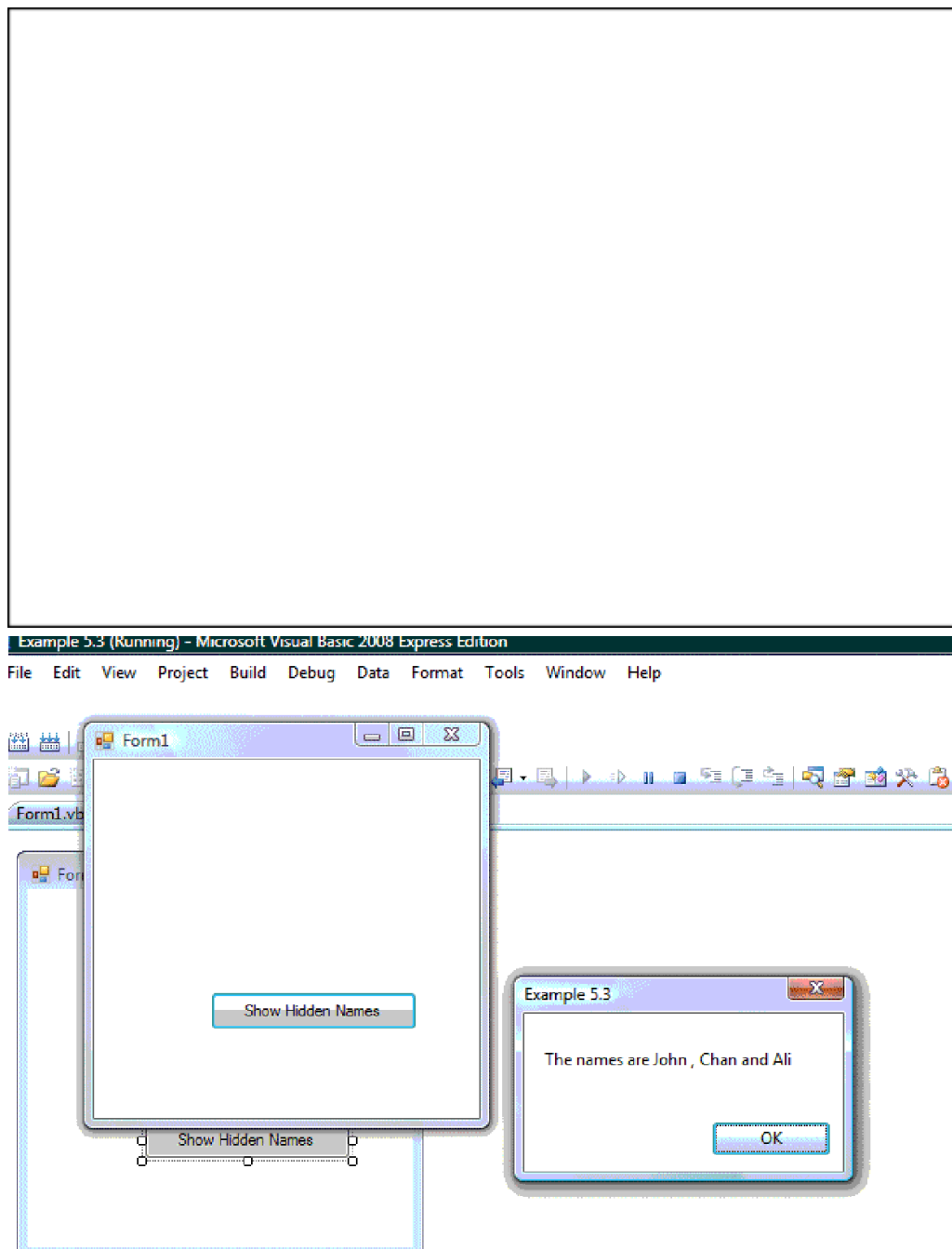
```
name3 = "Ali"
```

```
MsgBox(" The names are " & name1 & " , " & name2 & " and " & name3)
```

```
End Sub
```

In this example, you insert one command button into the form and rename its caption as Show Hidden Names. The keyword Dim is to declare variables name1, name2 and name3 as string, which means they can only handle text. The function MsgBox is to

display the names in a message box that are joined together by the "&" signs. The output is shown in Figure 5.4.



## Example 18.2

The screenshot shows a Windows application window titled "Form1". Inside the window, there are two panels. The left panel, titled "T-Shirt Color", contains three radio buttons: "Red" (which is selected), "Green", and "Yellow". The right panel, titled "T-Shirt Size", contains four radio buttons: "XL", "L", "M" (which is selected), and "S". Below these panels, there is a summary section. It starts with the text "You have chosen", followed by a text box containing the word "Red", then the word "color". Below this, it says "and size", followed by a text box containing the letter "M". To the right of this summary section is a button labeled "Confirm".

Figure 18.2

### The code

```
Dim strColor As String
```

```
Dim strSize As String
```

```
Private Sub RadioButton8_CheckedChanged(ByVal sender As System.Object, ByVal e
```

```
As System.EventArgs) Handles RadioButton8.CheckedChanged
strColor = "Red"
End Sub
```

```
Private Sub RadioButton7_CheckedChanged(ByVal sender As System.Object, ByVal e
As System.EventArgs) Handles RadioButton7.CheckedChanged
strColor = "Green"
End Sub
```

```
Private Sub RadioYellow_CheckedChanged(ByVal sender As System.Object, ByVal e
As System.EventArgs) Handles RadioYellow.CheckedChanged
strColor = "Yellow"
End Sub
```

```
Private Sub Button1_Click(ByVal sender As System.Object, ByVal e As
System.EventArgs) Handles Button1.Click
Label2.Text = strColor
Label4.Text = strSize
End Sub
```

```
Private Sub RadioXL_CheckedChanged(ByVal sender As System.Object, ByVal e As
System.EventArgs) Handles RadioXL.CheckedChanged
strSize = "XL"
End Sub
```

```
Private Sub RadioL_CheckedChanged(ByVal sender As System.Object, ByVal e As
System.EventArgs) Handles RadioL.CheckedChanged
strSize = "L"
End Sub
```

```
Private Sub RadioM_CheckedChanged(ByVal sender As System.Object, ByVal e As  
System.EventArgs) Handles RadioM.CheckedChanged  
strSize = "M"  
End Sub
```

```
Private Sub RadioS_CheckedChanged(ByVal sender As System.Object, ByVal e As  
System.EventArgs) Handles RadioS.CheckedChanged  
strSize = "S"  
End Sub
```

### Example 17.3

In this example, the user can enter text into a textbox and format the font using the three checkboxes that represent bold, italic and underline as shown in Figure 17.2

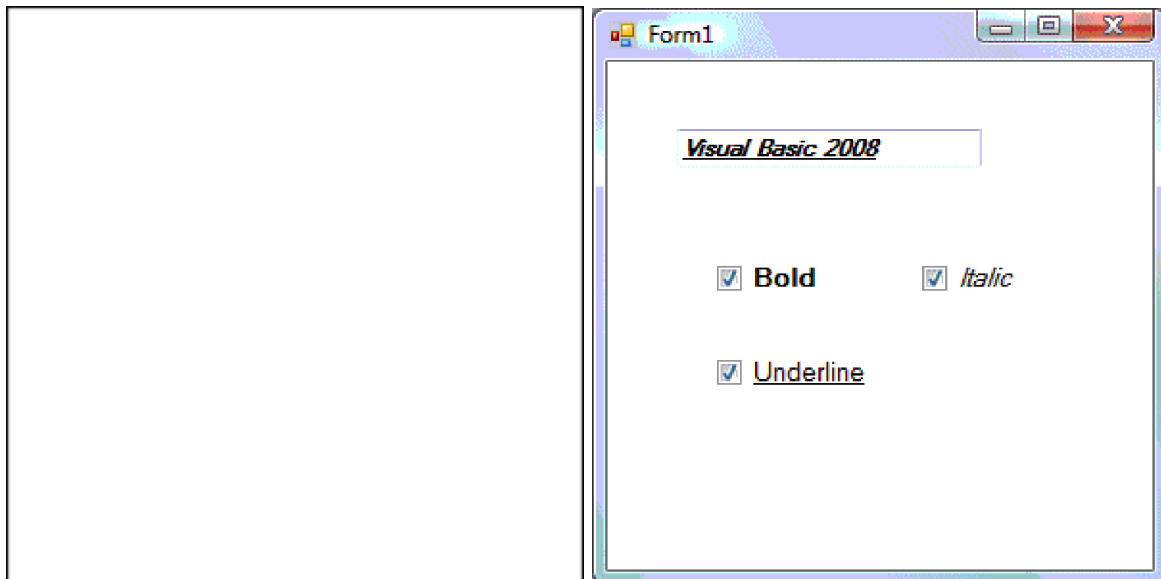


Figure 17.3

The code is as follow:

```
Private Sub CheckBox1_CheckedChanged(ByVal sender As System.Object, ByVal e As
```

System.EventArgs) Handles CheckBox1.CheckedChanged

If CheckBox1.Checked Then

TextBox1.Font = New Font(TextBox1.Font, TextBox1.Font.Style Or FontStyle.Bold)

Else

TextBox1.Font = New Font(TextBox1.Font, TextBox1.Font.Style And Not FontStyle.Bold)

End If

End Sub

Private Sub CheckBox2\_CheckedChanged(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles CheckBox2.CheckedChanged

If CheckBox2.Checked Then

TextBox1.Font = New Font(TextBox1.Font, TextBox1.Font.Style Or FontStyle.Italic)

Else

TextBox1.Font = New Font(TextBox1.Font, TextBox1.Font.Style And Not  
FontStyle.Italic)

End If

End Sub

Private Sub CheckBox3\_CheckedChanged(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles CheckBox3.CheckedChanged

If CheckBox3.Checked Then

TextBox1.Font = New Font(TextBox1.Font, TextBox1.Font.Style Or FontStyle.Underline)

Else

TextBox1.Font = New Font(TextBox1.Font, TextBox1.Font.Style And Not  
FontStyle.Underline)

End If

End Sub